

MACHINE LEARNING



K-Nearest Neighbors

Guessing the unknown by comparing it to what's
already similar



Table of content

01. Introduction to K-Nearest Neighbors (KNN)	_____	Motivation, intuition, and lazy learning
02. How KNN Works	_____	Algorithm steps, distance metrics, and choosing K
03. Practical Considerations	_____	Weighted KNN, feature scaling, decision boundaries, and hyperparameters
04. KNN in Practice	_____	Classification vs. Regression, Scikit-learn implementation, and model evaluation
05. Strengths, Limitations, and Performance	_____	Advantages, disadvantages, curse of dimensionality, computational complexity, and KDTree/BallTree

How would you guess a stranger's taste?

*“Imagine you meet someone for the first time.
How would you guess whether they like
football?”*

**Humans decide by
comparing new things with
similar things we've
already seen.**

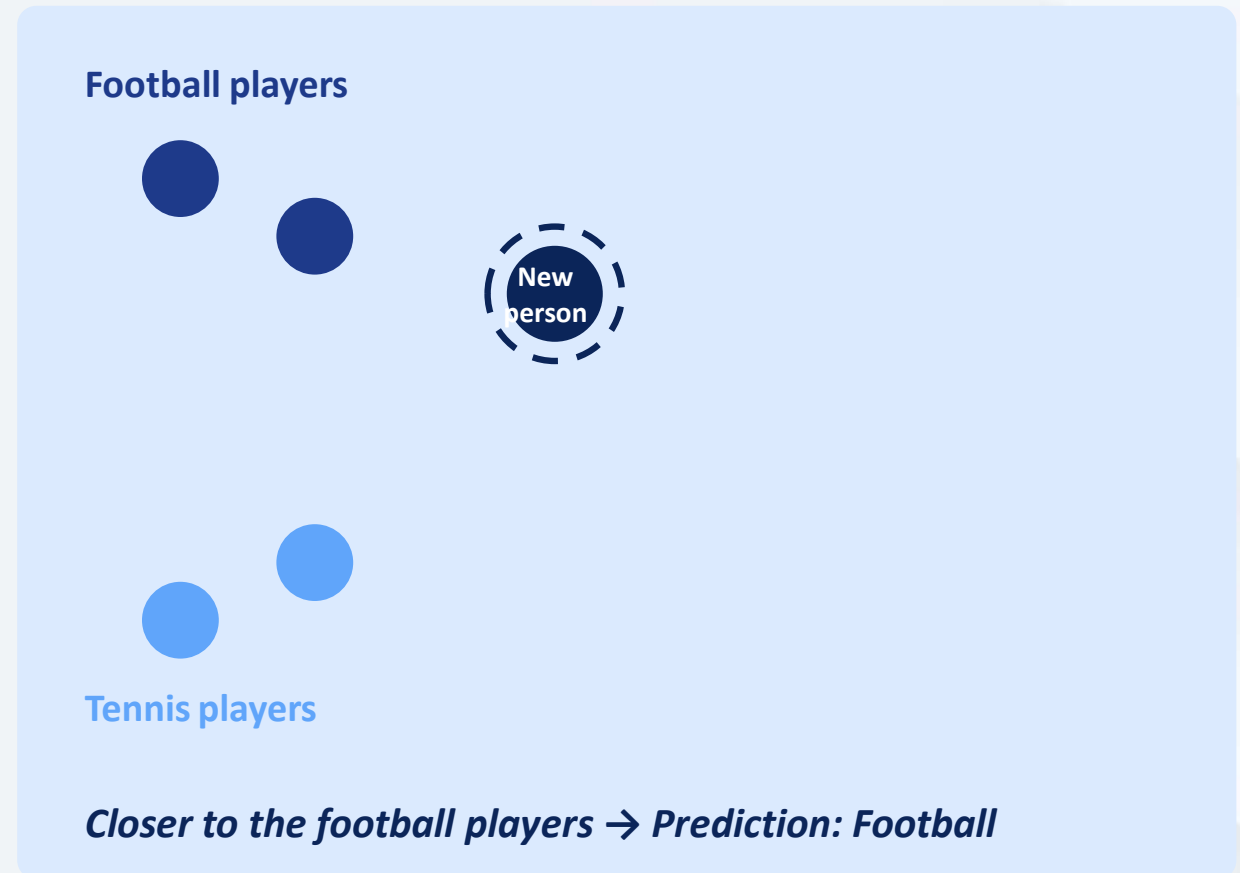
**KNN works exactly the
same way.**

A tiny example

Height	Weight	Sport
170	70	Football
168	68	Football
155	50	Tennis
158	52	Tennis

A new person arrives

Height = 169 Weight = 69



The key idea behind KNN

KNN doesn't learn complicated equations.

If two samples are similar, they probably belong to the same class.

Real-life examples students already know:

Netflix

Recommends movies liked by similar users

Amazon

"Customers similar to you bought this"

Spotify

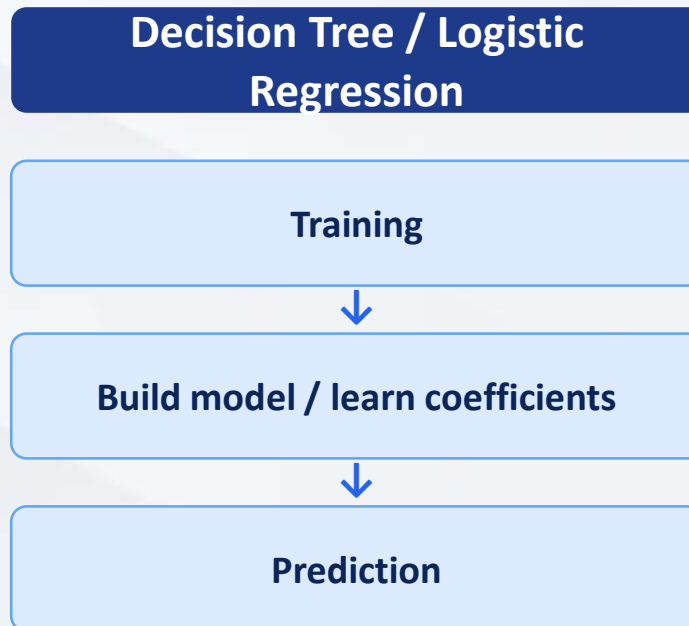
Suggests songs liked by similar listeners

Medicine

Similar symptoms → similar diagnosis

Does KNN build anything?

Most machine learning algorithms spend time learning from data before predicting.



No.

KNN simply stores the training data.

This is why it's called Lazy Learning.

The teacher analogy

Decision Tree teacher

Studies all semester.

On exam day → answers immediately.

KNN teacher

Never studies.

On exam day → opens every notebook, looks for similar examples, then answers.

Why KNN is “lazy”

No model is learned

No equations or trees are ever created.

Training is very fast

Training only means storing data — no optimization, no gradient descent.

Prediction is slower

Every new sample is compared against many (or all) training examples.

Memory usage is high

All training data is kept in memory, unlike models that store only parameters.

Three names, one idea

Lazy Learning

The algorithm postpones work until prediction time.

Instance-Based Learning

It predicts using individual training examples, not a general model.

Memory-Based Learning

It relies on storing training data in memory to make predictions.

All three describe the same general idea from different perspectives.

Step 1 & 2 — Store data, choose K

1 Store the training data

A → Football

B → Football

C → Tennis

D → Tennis

Nothing is learned yet.

2 Choose K

K = the number of neighbors we consider

K = 1 Look at only one neighbor

K = 3 Look at the three closest

K = 5 Look at the five closest

Step 3 & 4 — Distance and neighbors

3 Calculate distance

How do we know who is closest?

We measure the distance between the new sample and every training sample.

*The smaller the distance,
the more similar the two samples are.*

4 Find the K nearest neighbors

Suppose the three nearest neighbors are:

Football

Football

Tennis

Step 5 — Majority voting

Which class appears most among the K neighbors?

Football

2 votes

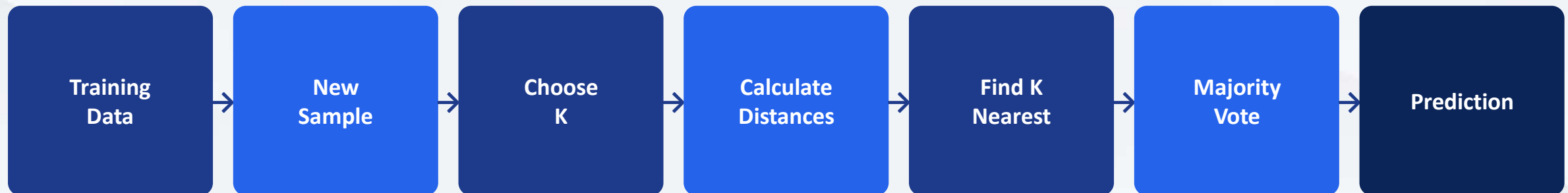
WINNER → Prediction: Football

Tennis

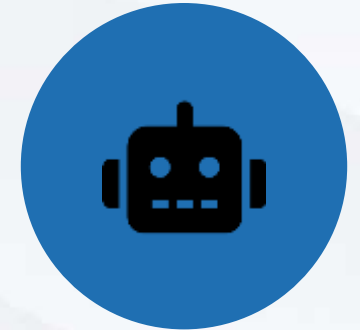
1 vote

For classification, KNN predicts using a majority vote among the nearest neighbors.

The complete KNN workflow



This visual summary is the whole algorithm — no equations, no trees, just comparison and a vote.



Distance Metrics in KNN

KNN says “find the nearest neighbors.”

But how does the computer know “nearest”?

A distance metric is simply a mathematical way to measure how similar or different two samples are.

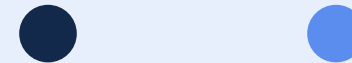
SMALLER DISTANCE

More similar



LARGER DISTANCE

Less similar



Euclidean Distance

"This is the distance we use in everyday life — the straight-line distance between two points."

Formula

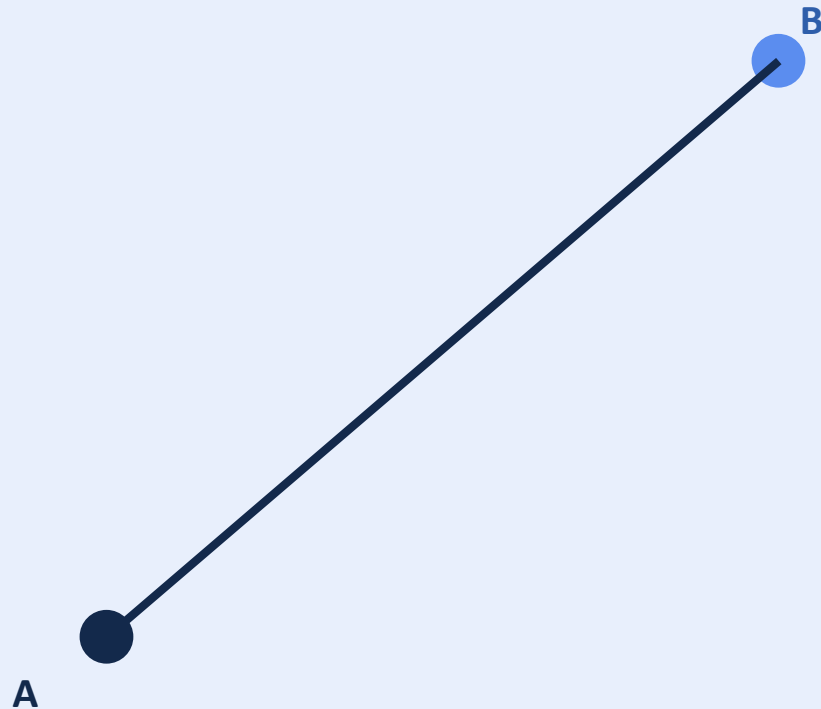
$$d = \sqrt{(\sum (x_i - y_i)^2)}$$

x_i = feature of the first sample

y_i = feature of the second sample

n = number of features

Square each difference → sum → square root

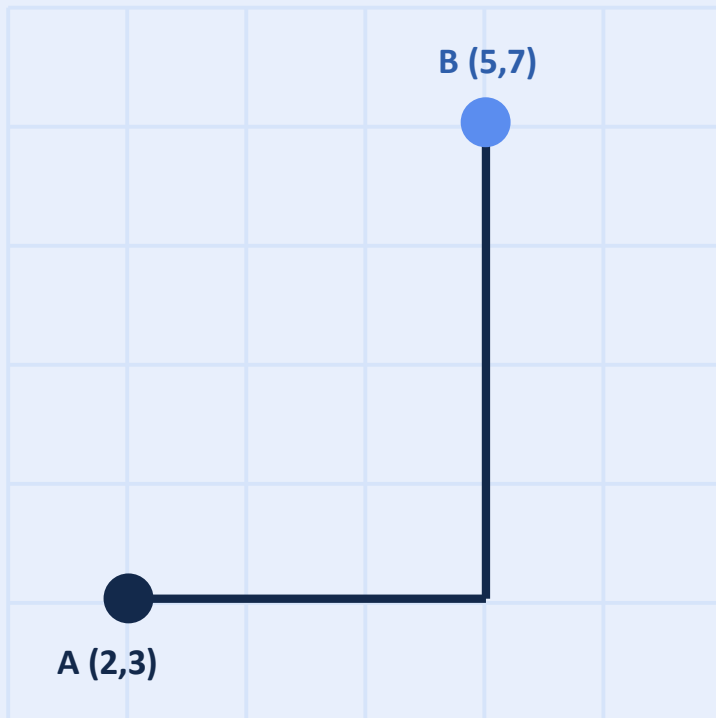


Person A (170, 70) Person B (168, 68)

$$d = \sqrt{(4 + 4)} = \sqrt{8} \approx 2.83$$

Manhattan Distance

“What if you're driving in a city? Can you drive diagonally through buildings?” — No.



Formula

$$d = \sum |x_i - y_i|$$

Instead of a straight line, you travel along the streets — useful whenever movement is constrained to grid-like paths.

$$\begin{aligned} A &= (2,3) \quad B = (5,7) \\ |5-2| + |7-3| &= 3 + 4 \\ d &= 7 \end{aligned}$$

Minkowski Distance

Euclidean and Manhattan are actually special cases of one general formula.

$$d = (\sum |x_i - y_i|^p)^{(1/p)}$$

$p = 1 \rightarrow$ Manhattan distance

$p = 2 \rightarrow$ Euclidean distance

This is exactly why scikit-learn exposes a p parameter:

```
KNeighborsClassifier(metric="minkowski", p=2)    # means Euclidean distance
```

Hamming Distance

“What if our features aren't numbers?” — e.g. gender, owns car, has laptop.

d = count of positions that differ

1	0	1	1	1	0
1	0	0	0	1	0
		▲	▲		

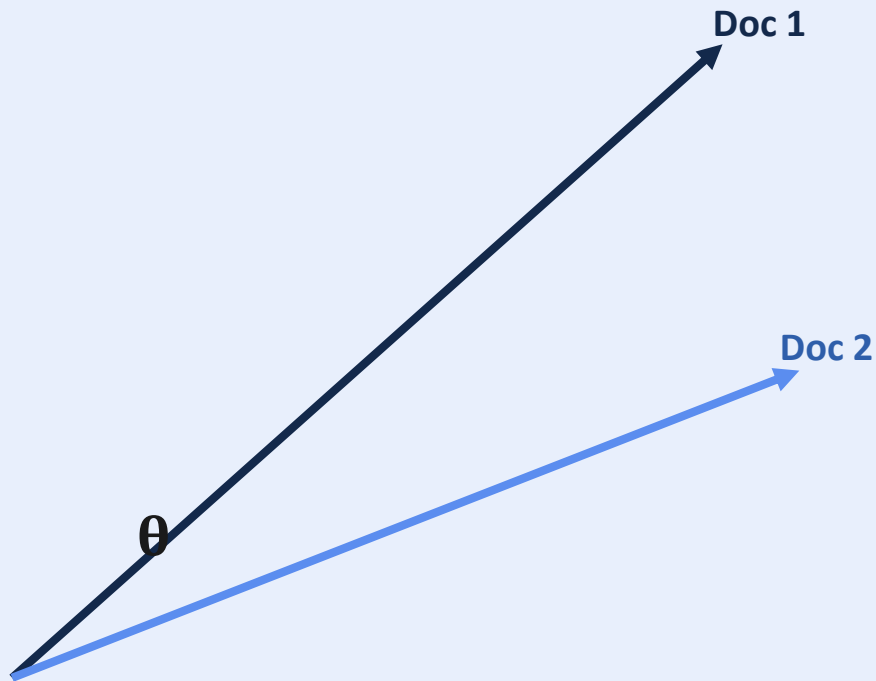
2 positions differ

Distance = 2

Commonly used for binary or categorical data.

Cosine Distance

"Sometimes the magnitude doesn't matter — only the direction does."



Document 1: AI, Machine Learning, Python

Document 2: AI, Deep Learning, Python

They share similar topics even though one document could be much longer.

Cosine similarity measures the angle between vectors — popular for text mining and NLP embeddings.

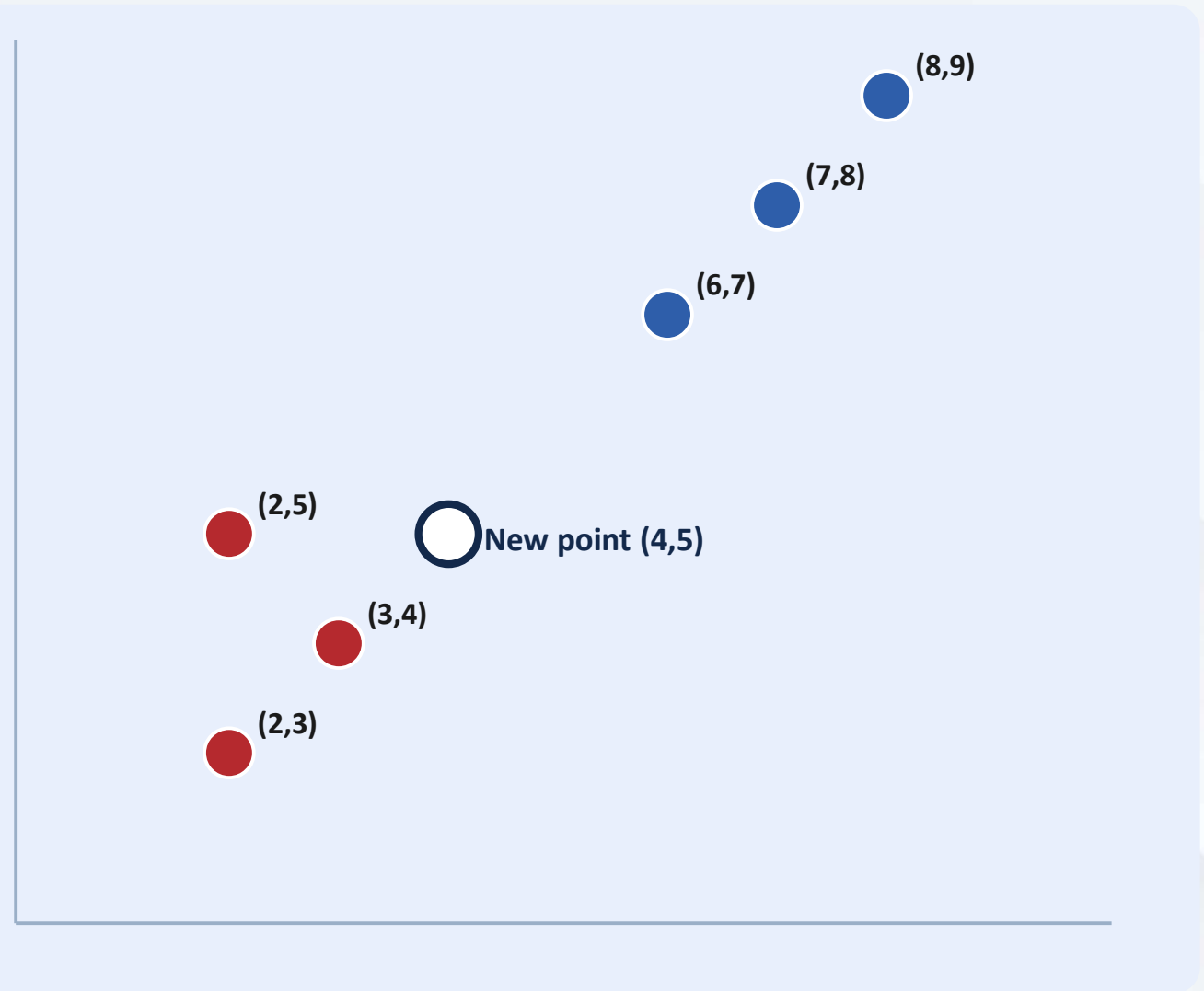
Which Metric Should I Use?

METRIC	BEST FOR
Euclidean	Continuous numerical features
Manhattan	Grid-like movement, robust to some outliers
Minkowski	Generalization of Euclidean and Manhattan
Hamming	Binary or categorical features
Cosine	Text data, document similarity, embeddings

Dataset

ID	X	Y	Class
1	2	3	Red
2	3	4	Red
3	7	8	Blue
4	8	9	Blue
5	2	5	Red
6	6	7	Blue

Step 1 — Choose $K = 3$



Compute, Sort, Vote

Step 2 — Distance from (4,5) to every point

Point	Distance
(2,3)	2.83
(3,4)	1.41
(7,8)	4.24
(8,9)	5.66
(2,5)	2.00
(6,7)	2.83

Step 3 — Sort by distance (K = 3 nearest)

Neighbor	Distance	Class
(3,4)	1.41	Red
(2,5)	2.00	Red
(2,3)	2.83	Red
(6,7)	2.83	Blue

Step 4 — Top 3 classes

Red, Red, Red

Step 5 — Majority vote

Prediction: Red

Choosing K

K is the major hyperparameter in KNN — and choosing it well is crucial.

SMALL K (e.g. $K = 1$)

The algorithm trusts only the closest neighbor.

✓ Very flexible, captures local patterns

✗ Highly sensitive to noise — can overfit

One mislabeled training point can flip the prediction entirely.

LARGE K (e.g. $K = 15$)

Many neighbors vote together.

✓ Smoother predictions, less noise-sensitive

✗ May ignore local structure — can underfit

Asking only your best friend is biased; asking the whole school is too general.

What Happens When Votes Tie?

$K = 4$, and the four nearest neighbors are: Red, Red, Blue, Blue. No majority — how is this handled?

1 Use an odd K

Instead of $K = 4$, choose $K = 3$ or $K = 5$ — this reduces the chance of ties in binary classification.

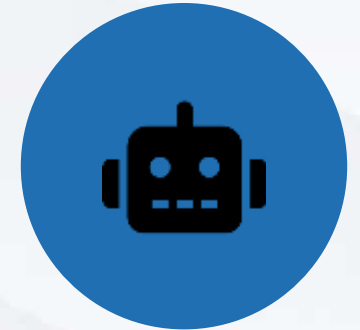
2 Weighted voting

Closer neighbors get more influence (weight $\approx 1/\text{distance}$), instead of every neighbor casting one equal vote.

3 Library-specific behavior

scikit-learn's default (weights='uniform') resolves ties by label order; weights='distance' often avoids ties entirely.

Weighted voting example — Blue wins on total weight despite a 2–2 split:



How Do We Choose the Best Value of K ?

There Is No Formula for K

K is a hyperparameter — it has to be tuned, not calculated

There is **no mathematical formula** that tells us the perfect K.

Instead, we **try several values of K** and **evaluate the model** on each one.

1

Train

Fit KNN with several candidate K values

2

Measure

Evaluate accuracy on a validation set

3

Choose

Pick the K with the best validation performance

Step 1 — Train KNN With Different K Values

Pick a range of candidate K values to test



Only odd values of K are used here to avoid tie votes in binary classification.

For each value of K:

- Train a separate KNN model using that K
- Evaluate it on validation data (never on training data)

Step 2 — Measure the Performance

Record validation accuracy for every K tested

K	Accuracy
1	96%
3	98%
5	99%
7	98%
9	97%
11	96%
13	95%

Best result

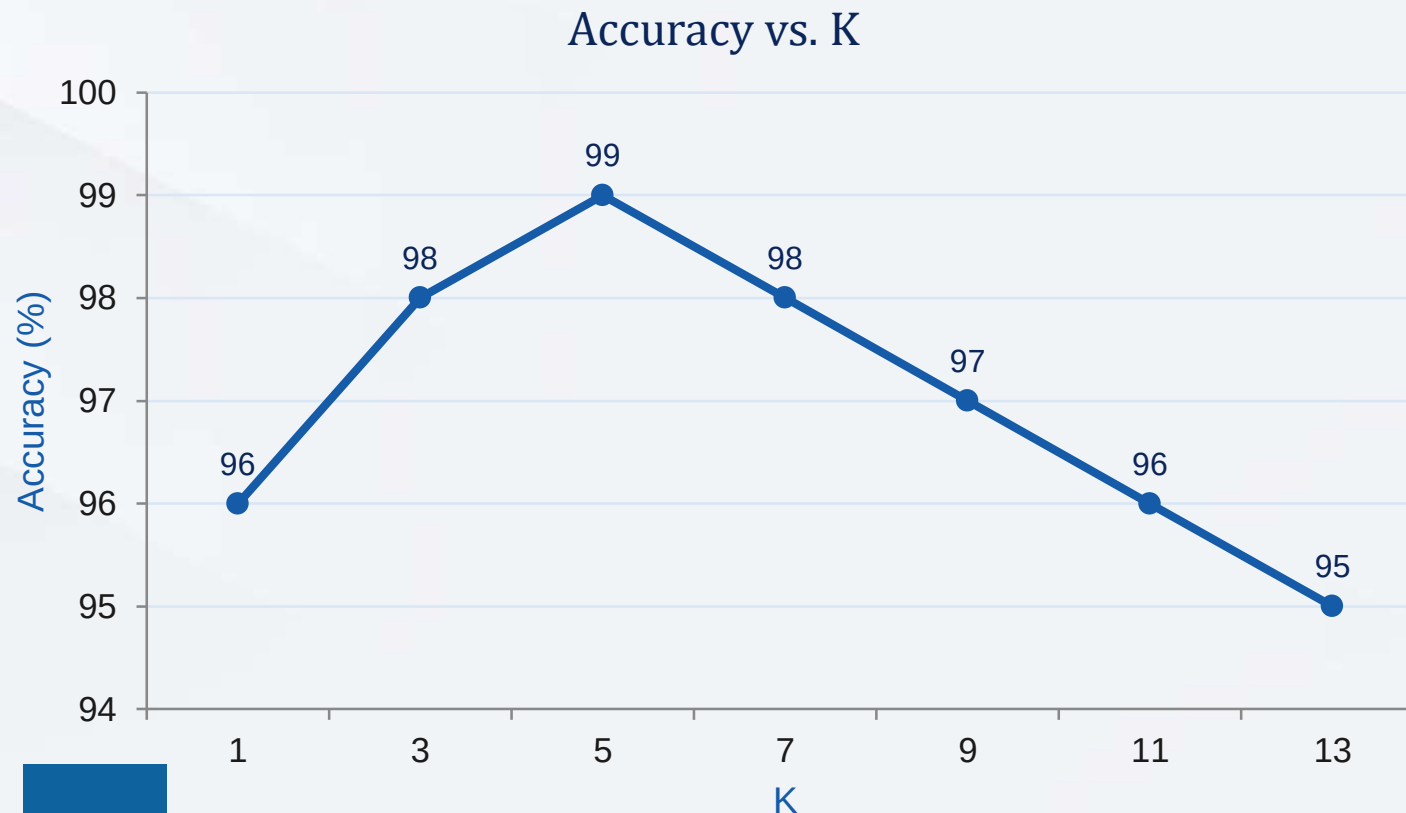
K = 5

reaches **99% accuracy** — the highest of all K values tested.

Accuracy rises from K=1 to K=5, then gradually falls as K keeps growing.

Step 3 — Plot Accuracy vs. K

Choose the K that gives the highest validation accuracy



Rule of thumb

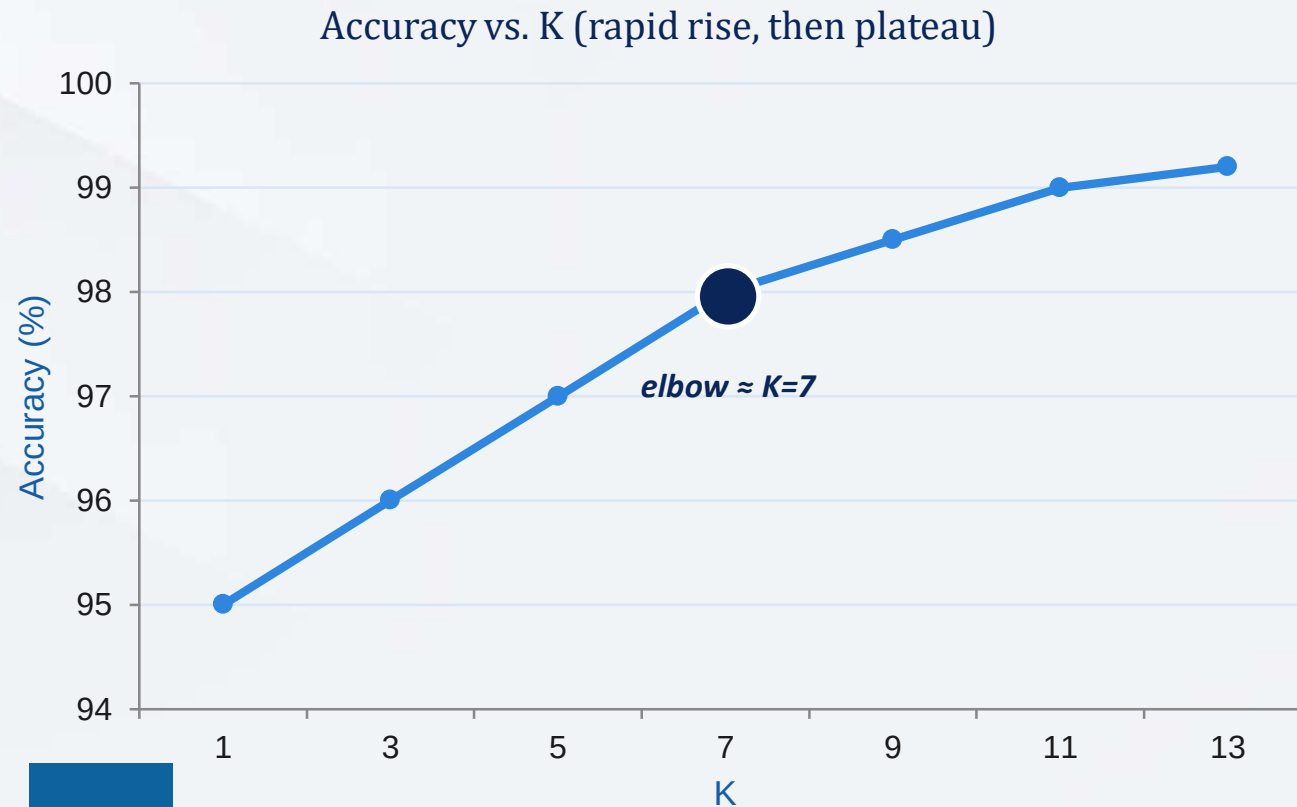
"We usually choose the K that gives the highest validation accuracy."

In this example: **K = 5**

Peak accuracy, then the curve declines as K keeps increasing.

What About the “Elbow”?

Sometimes accuracy rises fast, then levels off



The “elbow” point

Some call the point where improvement starts to slow down the elbow.

“Beyond this point, increasing K doesn’t significantly improve the model.”

Important Clarification

This is NOT the same Elbow Method used in K-Means

K-Means Clustering

The classic Elbow Method

X-axis: Number of clusters (K) **Y-axis:** WCSS (Within-Cluster Sum of Squares) **Goal:** find where WCSS stops dropping sharply

This is the standard, well-established “elbow method.”

KNN Classification

What we actually plot here

X-axis: Value of K **Y-axis:** Validation accuracy (or error) **Goal:** find the K with the best validation score

More accurately called hyperparameter tuning, not the Elbow Method.

Simple Code — Searching for the Best K

Don't forget the elbow search: loop over K, don't hardcode it

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

best_k, best_score = None, 0

for k in range(3, 15, 2): # try odd K values
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5)
    avg_score = scores.mean()

    if avg_score > best_score: # keep the best K
        best_k, best_score = k, avg_score

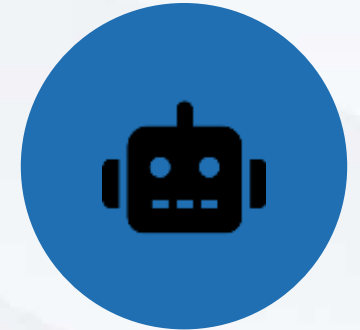
print("Best K:", best_k)
```

Don't forget

K is never fixed by a formula.

The `for k in range(...)` loop is the elbow / best-K search.

Never hardcode a single K — always search and compare validation scores first.



KNN Regression

Can KNN Predict a Continuous Value?

Like house price or salary — most students think KNN is only for classification.

YES — the only difference is how the prediction is made.

CLASSIFICATION

Output: a class
(Red / Blue, Cat / Dog)

Prediction via Majority Voting

REGRESSION

Output: a number
(House Price, Salary, Grade)

Prediction via Average / Weighted Average

Instead of Voting, We Average

EXAMPLE 1 — HOUSE PRICE

3 nearest houses cost:

\$120,000

\$140,000

\$150,000

$$(120+140+150) / 3$$

$\approx \$136,667$

EXAMPLE 2

Neighbor values:

12

14

15

$$(12+14+15) / 3$$

≈ 13.67

Weighted KNN Regression

“Should a house that's only 100 meters away influence the prediction the same as one that's 5 km away?”

No.

So instead of a simple average, we use a weighted average — closer neighbors contribute more.



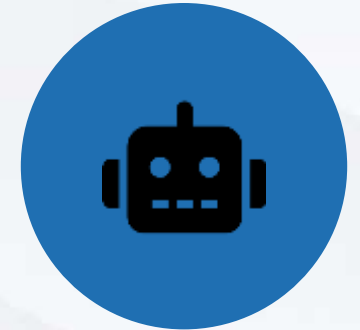
Classification vs. Regression

	Classification	Regression
Predicts	A class	A number
Method	Majority vote	Average / weighted average
Example	Spam or not spam	House price

scikit-learn — the algorithm is almost identical; only the final prediction step changes.

```
from sklearn.neighbors import  
KNeighborsClassifier
```

```
from sklearn.neighbors import  
KNeighborsRegressor
```



Hyperparameters & Complexity

n_neighbors — The Most Important One

KNN is simple, but its performance depends heavily on a few hyperparameters.

```
KNeighborsClassifier(n_neighbors=5)
```

SMALL K

More flexible

More sensitive to noise

Overfitting

LARGE K

Smoother boundary

More stable

Underfitting

“What happens if K equals the entire training dataset?” — every prediction becomes the majority class.

metric & weights

METRIC — how distance is calculated

Euclidean	Numerical features
Manhattan	Grid-like movement
Hamming	Binary features
Cosine	Text similarity

WEIGHTS — how votes are counted

`weights="uniform"`

Every neighbor gets one vote.

`weights="distance"`

Closer neighbors have greater influence.

If one neighbor is much closer than the others, use distance.

algorithm, leaf_size & p

algorithm



How KNN searches for neighbors: auto, ball_tree, kd_tree, brute. scikit-learn usually chooses automatically.

leaf_size



Only affects BallTree/KDTree construction.

p



Used only with metric="minkowski". $p=1 \rightarrow$ Manhattan. $p=2 \rightarrow$ Euclidean.

KDTree and BallTree

Problem: searching through every point is slow.

Solution: organize the data so we don't check every sample.

*Searching for a word — without an index, you read every page; with an index, you jump directly to the right section.
KDTree and BallTree are like indexes for your training data.*

KDTree

Partitions space using axis-aligned splits. Works well for low- to moderate-dimensional numerical data.

BallTree

Groups points into nested hyperspheres (“balls”).
Can be more efficient for higher dimensions or certain metrics.

```
KNeighborsClassifier(algorithm="auto") # scikit-learn picks automatically
```

Hyperparameter Summary

Hyperparameter	Purpose
<code>n_neighbors</code>	Number of neighbors
<code>metric</code>	Distance function
<code>weights</code>	Uniform or distance voting
<code>algorithm</code>	Search strategy
<code>leaf_size</code>	Tree optimization
<code>p</code>	Minkowski exponent

Why KNN Is a “Lazy Learner”

TRAINING

$O(1)$

Training just stores the dataset — no optimization, no tree building, no gradient descent.

Negligible training cost.

PREDICTION

$O(n)$

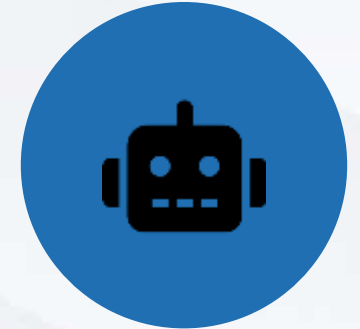
Every new sample is compared with every training example — 1,000,000 samples $\approx$ 1,000,000 distance calculations.

Prediction slows as the dataset grows.

Memory Cost & Algorithm Comparison

Does KNN keep only a few parameters, like Logistic Regression? No — it stores every training sample, so memory usage is high.

Algorithm	Training	Prediction
Decision Tree	Slow	Fast
Logistic Regression	Slow	Fast
Neural Network	Very Slow	Fast
KNN	Very Fast	Slow



Advantages, Limitations & Practical Implementation

Advantages

**“If KNN has so many limitations,
why do people still use it?”**

Advantages of KNN

1

Simple

The easiest ML algorithm to understand — no equations to optimize, no complex training. Just compare with neighbors.

2

No Training Phase

KNN simply stores the data. Training is almost instantaneous — unlike a Decision Tree or Logistic Regression, which learn a structure or parameters.

3

Easy to Understand

Calculate distance → find neighbors → vote. Students can perform it by hand — often the first classifier taught in ML.

Advantages of KNN (cont.)

4

Handles Nonlinear Boundaries

Every prediction depends on nearby samples, so KNN can trace irregular boundaries — e.g. a spiral dataset, where Logistic Regression struggles.

5

Good for Small Datasets

With only a few hundred samples, prediction stays cheap and KNN performs very well.

6

Naturally Supports Multiclass

Majority vote works whether there are 2, 3, or 10 classes — no modifications needed, unlike Logistic Regression's binary origins.

Disadvantages

“Now let’s see why KNN isn’t always the best choice.”

Disadvantages of KNN

1

Slow Prediction

Every new sample requires computing many distances. With millions of samples, prediction becomes too slow for real-time systems.

2

Requires Feature Scaling

If one feature ranges 1–10 and another 1–100,000, the larger feature dominates the distance. Always scale numerical features first.

3

Sensitive to Irrelevant Features

An irrelevant feature (e.g. House ID) distorts distance calculations. KNN treats every feature as important unless you remove or weight it.

Disadvantages of KNN (cont.)

4

Curse of Dimensionality

As feature count grows, distances stop being meaningful. Covered in depth on the next slides.

5

High Memory Usage

Unlike a Decision Tree, KNN stores every training sample — large datasets require much more RAM.

6

Choosing K Isn't Easy

$K = 1$ isn't always best, and neither is $K = 100$. The optimal K depends on the dataset — try several values with cross-validation.

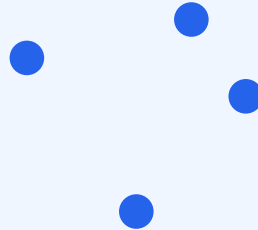
The Curse of Dimensionality

Imagine you're looking for your friend...



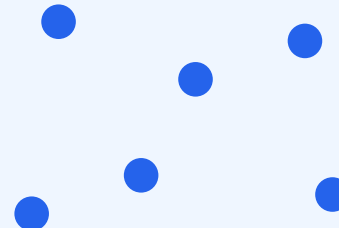
1 Dimension

A street



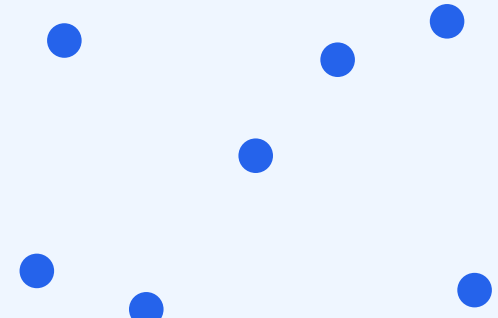
2 Dimensions

A city



3 Dimensions

A building



100+ Dimensions

A vast universe

If every point is far from every other point, does “nearest neighbor” still mean much?

Why Does KNN Suffer?

KNN relies entirely on distance.

If distance becomes meaningless, the algorithm loses its ability to identify similar samples.

How do we solve it?

Feature Selection

PCA

Dimensionality Reduction

Collect More Data (sometimes)

Avoiding Data Leakage

“Why don’t we simply do this?”

Wrong

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.fit_transform(X_test)
```

fit_transform on the test set computes mean/std from test data — leaking information into the model.

Correct

```
pipeline = Pipeline([  
    ("scaler", StandardScaler()),  
    ("knn", KNeighborsClassifier())  
)  
pipeline.fit(X_train, y_train)
```

Fits the scaler once on training data, then applies the same transform automatically at prediction time.

Pipeline prevents data leakage and keeps preprocessing + model bundled together.

Model Evaluation

“How do we know if our model is good?”

Accuracy

Correct Predictions / Total Predictions

90 correct out of 100 → 90% accuracy.

But with imbalanced data, accuracy alone can mislead.

	Predicted Cat	Predicted Dog
Actual Cat	40	3
Actual Dog	2	55

Precision, Recall & F1

Precision

When the model predicts
Positive, how often is it
correct?

Spam detection · Fraud detection

Recall

Of all actual Positive cases,
how many did we find?

Cancer diagnosis · Disease detection

F1-Score

The harmonic mean of
Precision and Recall.

*Balanced measure for imbalanced
classes*

Choosing K

for k in [1, 3, 5, 7, 9]:

K	Accuracy
1	0.93
3	0.96
5	0.98
7	0.97
9	0.95

“Which K would you choose?”

K = 5 — it gives the best validation/test performance.

Coding KNN — Classification vs. Regression

Same algorithm, two different scikit-learn classes

Classification

Predicts a category (label)

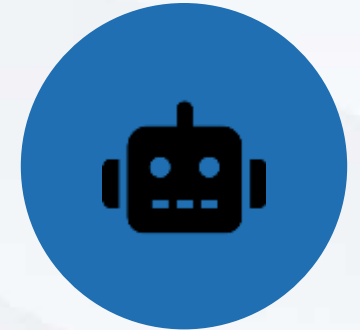
```
from sklearn.neighbors import  
    KNeighborsClassifier  
  
clf = KNeighborsClassifier(n_neighbors=5)  
clf.fit(X_train, y_train)  
  
y_pred = clf.predict(X_test)  
# e.g. "cat" / "dog"  
  
acc = clf.score(X_test, y_test)  
print("Accuracy:", acc)
```

Regression

Predicts a continuous number

```
from sklearn.neighbors import  
    KNeighborsRegressor  
  
reg = KNeighborsRegressor(n_neighbors=5)  
reg.fit(X_train, y_train)  
  
y_pred = reg.predict(X_test)  
# e.g. 5 nearest  
# prices, averaged  
  
r2 = reg.score(X_test, y_test)  
print("R2:", r2)
```

Key difference: Classifier votes on the majority label; Regressor averages the neighbors' values.



Lecture Summary

Concept	Key Idea
KNN	Predict using the nearest neighbors
Distance	Defines similarity
K	Controls model complexity
Scaling	Essential for fair distance calculations
Classification	Majority vote
Regression	Average (or weighted average) of neighbors

Weighted KNN	Closer neighbors have more influence
Advantages	Simple, intuitive, no training, nonlinear boundaries
Disadvantages	Slow prediction, memory intensive, sensitive to scaling and dimensions
Best Practice	Scale features, tune K with cross-validation, use a Pipeline

Thank you